



MongoDB

MongoDB Introduction?



- MongoDB is an open-source document database and leading NoSQL database. It is written in C++.
- It stores the data in a type of JSON format called **BSON**.
- It works on concept of collection and document.
- **Database:** Database is a physical container for collections. A single MongoDB server typically has multiple databases.
- **Collection:** Collection is a group of MongoDB documents. It is the equivalent of an RDBMS table.
 - A collection exists within a single database.
 - Documents within a collection can have different fields.
- **Document:** A document is a set of key-value pairs. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields.

MongoDB Document



- MongoDB document is a simple comma separated key value pair.

```
1 {  
2   "name": "Jhon",  
3   "department": "Marketing",  
4   "designation": "Manager",  
5   "qualification": "MBA",  
6   "salary": "$2000",  
7   "organization": "DM Solutions"  
8 }
```



Connect Node with MongoDB

- For help visit **mongodb npm** official website.
- Create a node project using command **npm init**
- Install express using command **npm i --save express**
- Install mongodb driver using command **npm i --save mongodb**
- Now create a file **mongodb.js** inside the node project for database connection.
- Create **index.js** file to create a get api which will fetch the data from the database.

mongodb.js



```
const { MongoClient } = require("mongodb");
const url = "mongodb://localhost:27017";
const databaseName = "test-db";
const client = new MongoClient(url);

async function dbConnect() {
  let result = await client.connect();
  db = result.db(databaseName);
  return db.collection("students");
}

module.exports = dbConnect;
```



Read the data From MongoDB

```
const express = require("express");
const dbConnect = require("./mongodb");
const app = express();
app.use(express.json());

app.get("/", async (req, res) => {
  let data = await dbConnect();
  data = await data.find().toArray();
  res.send(data);
});

app.listen(8080);
```

To find the specific data from database replace this statement with

→
`data = await data.find({
 name: "Mohit"
}).toArray();`

Now open the postman and hit the get api: <http://localhost:8080/>



Insert data Into MongoDB

- Create a **insert.js** file inside the node project.
- Import database connection into this **insert.js** file.
- Use a middleware `express.json()`
- The final file will look like

```
const express = require("express");
const dbConnect = require("../mongodb");
const app = express();
app.use(express.json());
app.post("/", async (req, res) => {
    let data = await dbConnect();
    let result = await
data.insertOne(req.body);
    res.send(result);
});
app.listen(8080);
```

Update data Into MongoDB



- Create a **update.js** file inside the node project.
- Use a middleware `express.json()`
- Use **updateOne()** method to update the data.
- The final file will look like

```
const express = require("express");
const dbConnect = require("../mongodb");
const app = express();
app.use(express.json());

app.put("/", async (req, res) => {
  let data = await dbConnect();
  let result = await data.updateOne(
    { name: req.body.name },
    { $set: req.body }
  );
  res.send(result);
});

app.listen(5000);
```


Update data Into MongoDB



- We can also pass the parameters from the postman.
- The postman request will look like:

<http://localhost:5000/postman>

```
const express = require("express");
const dbConnect = require("../mongodb");
const app = express();
app.use(express.json());

app.put("/:name", async (req, res) => {
  let data = await dbConnect();
  let result = await data.updateOne(
    { name: req.params.name },
    { $set: req.body }
  );
  res.send(result);
});

app.listen(5000);
```



Delete data From MongoDB

- We will delete the data by using the id of record.
- Import the mongodb and create the object of Id.
- The postman request will look like:

<http://localhost:5000/661910e8b2799d937c81b7d7>

```
const express = require("express");
const mongodb = require("mongodb");
const dbConnect = require("../mongodb");
const app = express();
app.use(express.json());
```

```
app.delete("/:id", async (req, res) => {
  let data = await dbConnect();
  let result = await data.deleteOne({
    _id: new
mongodb.ObjectId(req.params.id),
  });
  res.send(result);
});
```

```
app.listen(5000);
```



Get Data From API and Display in Angular Component



Use get API in Angular Component

- Create a service named **studentsData** in your angular project.

ng g s services/studentsData

- Import HttpClient in this service file

```
import { HttpClient } from '@angular/common/http';
```

- Create and initialize the instance variable of HttpClient inside the constructor.

```
constructor(private http: HttpClient) {}
```

- Create a function **students()** and return the data from this method

```
students() {  
    return this.http.get('http://localhost:8080/students');  
}
```



Use get API in Angular Component

- Import ***HttpClientModule*** inside the ***app.module.ts*** file and register in ***imports*** array.

```
import { HttpClientModule } from '@angular/common/http';
```

- The final students-data.service.ts file will look like

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
@Injectable({
  providedIn: 'root',
})
export class StudentsDataService {
  constructor(private http: HttpClient) {}
  students() {
    return this.http.get('http://localhost:8080/students');
  }
}
```



Use get API in Angular Component

- Create a component where you want to use this service and put the following code in **ts** file.

```
import { Component } from '@angular/core';
import { StudentsDataService } from '../services/students-data.service';
@Component({
  selector: 'app-header',
  templateUrl: './header.component.html',
  styleUrls: ['./header.component.css'],
})
export class HeaderComponent {
  data: any;
  constructor(student: StudentsDataService) {
    student.students().subscribe((res) => {
      this.data = res;
    });
  }
}
```


Use get API in Angular Component



- Inside the html file of the component write the following code.

```
<h1>Students Data</h1>
<table class="table">
  <thead>
    <tr>
      <th>Name</th>
      <th>Branch</th>
      <th>City</th>
      <th>Age</th>
    </tr>
  </thead>
  <tbody>
```

```
    <tr *ngFor="let user of data">
      <td>{{user.name}}</td>
      <td>{{user.branch}}</td>
      <td>{{user.city}}</td>
      <td>{{user.age}}</td>
    </tr>
  </tbody>
</table>
```



Use Post API to Insert the Data

- Create a new component called ***create-student***
- Create a form inside this component.

```
<div style="margin-left: 40px;">
  <form class="form" #studentsForm="ngForm"
    (ngSubmit)="getStudentsFormData(studentsForm.value)">
    <input type="text" name="name" ngModel
      placeholder="Enter Name"><br><br>
    <input type="number" name="age" ngModel
      placeholder="Enter Age"><br><br>
    <input type="text" name="gender" ngModel
      placeholder="Enter Gender"><br><br>
    <input type="text" name="branch" ngModel
      placeholder="Enter Branch"><br><br>
    <button>Add New Student</button>
  </form>
</div>
```



Use Post API to Insert the Data

- Import **FormsModule** in app.module.ts file.
- Now in component ts file create the `getStudentsFormData()` function

```
import { Component } from '@angular/core';
import { StudentsDataService } from
'../services/students-data.service';

@Component({
  selector: 'app-create-student',
  templateUrl: './create-student.component.html',
  styleUrls: ['./create-student.component.css'],
})
export class CreateStudentComponent {
  constructor(private student:
StudentsDataService) {}
  getStudentsFormData(data: any) {
    this.student.students(data).subscribe((result)
=> {
      console.log(result);
    });
  }
}
```



Use Post API to Insert the Data

- Now inside the service file make the following changes.

```
import { Injectable } from '@angular/core';
import { HttpClient } from
 '@angular/common/http';

@Injectable({
  providedIn: 'root',
})
export class StudentsDataService {
  constructor(private http: HttpClient) {}
  students(data: any) {
    return
    this.http.post('http://localhost:8080/studen
    ts', data);
  }
}
```



Use Post API to Insert the Data

- Sometimes node may throw error. To solve this error use the middleware ***cors***.
- To use ***cors*** first install cors using command ***npm install cors***.

- Import cors

```
let cors = require("cors");
```

- Use core

```
app.use(cors());
```